

Project Brief

Interpersonal AI Firewall

Working Title

Interpersonal AI Firewall, or IAF

Core Idea

Interpersonal AI Firewall is an open source behavioral safety middleware layer for personal AI assistants.

The first implementation should be a local-first software tool for stressful communication across text, email, chat, voicemail transcription, and call-screening workflows.

The long-term vision is larger. Over a 15 to 20 year timescale, IAF should evolve into a consent-aware behavioral safety layer for personal AI agents, voice assistants, smart glasses, wearable AI devices, home robots, and future humanoid assistant robots.

IAF is designed to help users manage high-conflict, emotionally stressful, confusing, or pressure-filled communication without being pulled into instant escalation. It extracts facts, flags urgent content, drafts calm boundary responses, recommends reply delays when appropriate, and preserves a local tamper-evident communication ledger.

The system should not diagnose people, label anyone as abusive, infer malicious intent, or decide who is right. It should classify observable communication patterns and support the user's attention, boundaries, safety, and decision-making process.

The repo should be designed from day one as software-first, robotics-ready, and infrastructure-oriented.

One-Line Summary

Interpersonal AI Firewall is a local-first behavioral safety layer for personal AI assistants that turns stressful communication into facts, calm replies, delay recommendations, and auditable records, with a long-term path toward worldwide AI assistant and humanoid robot deployment.

Long-Term Infrastructure Thesis

Future personal AI assistants will not only answer questions. They will increasingly act as intermediaries between people and the outside world.

A personal AI assistant may eventually:

- Screen stressful messages.
- Summarize tense emails.
- Help users respond to hostile communication.
- Transcribe and buffer phone calls.
- Support co-parenting communication.
- Help an overwhelmed user pause before responding.
- Answer the door as a home robot.
- Speak user-approved boundary responses.
- Take messages during stressful encounters.
- Escalate urgent issues immediately.
- Maintain clear records of what happened.

This creates a major safety problem.

When AI assistants represent people in emotional, legal, family, medical, financial, workplace, or physical interactions, they need a behavioral firewall.

IAF is intended to become that firewall.

The first version is a practical local software tool. The long-term target is a trusted middleware layer, SDK, protocol, and governance framework for how personal AI assistants should behave when human interaction becomes stressful, urgent, manipulative, confusing, or high-pressure.

IAF should not try to become a general operating system. Instead, it should aim to become a behavioral safety layer that can be embedded across many platforms.

The long-term commercial vision is to become infrastructure for:

- Personal AI agents.
- AI assistant platforms.
- Communication tools.
- Legal-adjacent workflows.
- Mediation and co-parenting tools.
- Mental wellness adjacent tools.
- Elder care and caregiving systems.
- Smart home systems.
- Voice assistants.
- Smart glasses.
- Wearable AI.
- Humanoid assistants.
- Home robots.
- Robotics companies.
- Safety-focused AI platforms.

The 15 to 20 year goal is for IAF to become a recognized trust layer for user-controlled AI mediation of stressful human interaction.

Product Thesis

AI assistants are becoming more capable, but capability alone is not enough.

A personal assistant that can speak, reply, summarize, schedule, call, text, or operate inside a robot body needs clear behavioral boundaries.

The assistant must know:

- What must be surfaced immediately.
- What can be summarized.
- What should not be answered yet.
- What requires user approval.
- What should be logged.
- What should never be said.
- What should never be hidden.
- What must be escalated.
- What should be routed to a human professional.
- How to avoid impersonation.
- How to avoid conflict escalation.
- How to respect consent.
- How to behave safely when embodied in a robot.

IAF should provide this layer.

The core product is not “an app for empaths.”

The larger product is:

Consent-aware behavioral safety middleware for personal AI agents and embodied assistants.

The empathic and emotionally sensitive user base is the first human-centered use case. The infrastructure opportunity is broader.

Core Product Philosophy

IAF gives the user a pause.

It turns chaos into facts.

It turns emotional drafts into calm boundaries.

It turns confusing histories into searchable records.

It protects the user's attention without replacing the user's judgment.

It helps AI assistants behave with restraint.

It makes user approval, consent, auditability, and de-escalation core system behaviors.

In the long term, it gives future personal humanoid assistants a behavioral framework for staying steady, factual, calm, transparent, and respectful during stressful real-world interactions.

Target Users

IAF is especially useful for people who experience communication overload during tense interactions.

Initial users include:

- Emotionally sensitive users.
- People who freeze, overexplain, or spiral under pressure.
- People navigating hostile co-parenting communication.
- People managing difficult family communication.
- People recovering from toxic or high-conflict relationships.
- People who need factual records of stressful exchanges.
- People who want help drafting short, calm, non-escalating replies.
- People who want a trusted assistant between themselves and stressful inbound messages.

Professional and institutional users may later include:

- Mediators.
- Family law professionals.
- Legal aid organizations.
- Therapists and counselors.
- Domestic violence support organizations.
- Elder care coordinators.
- Care managers.
- School administrators.
- HR professionals.
- Conflict resolution professionals.
- Co-parenting platforms.
- Communication platforms.
- Robotics companies.
- AI agent platforms.

The repo should use neutral, credible language. It can mention emotionally sensitive users and high-conflict communication, but it should avoid sounding like a diagnostic tool, legal advisor, abuse detector, or therapy replacement.

Commercial Category

The safest commercial category is:

Communication safety and documentation software for AI-assisted interaction.

The larger long-term category is:

Behavioral safety middleware for personal AI agents and embodied assistants.

The product should not be positioned as:

- AI legal advisor.
- AI therapist.
- AI abuse detector.
- AI lie detector.
- AI judge of intent.
- Autonomous legal evidence system.
- Robot bodyguard.
- Robot police proxy.
- Fully autonomous conflict agent.

Commercially, IAF should remain legal-adjacent, not a legal advisor.

The product may help users organize, summarize, de-escalate, and document communications. It must not determine legal rights, provide legal strategy, diagnose people, or decide whether something is legally admissible.

15 to 20 Year Roadmap

Stage 1: Open Source Core

Build a local-first open source package.

Goals:

- Establish the architecture.
- Build trust.

- Show safety principles.
- Create testable core logic.
- Support CLI workflows.
- Include LLM adapters.
- Include mock adapters.
- Include local ledger.
- Include humanoid assistant stubs.

Primary output:

A working GitHub repo that proves the core behavioral firewall.

Stage 2: Local Consumer Tool

Create a privacy-first local app.

Possible forms:

- Desktop app.
- Browser extension.
- Local web app.
- Mobile companion app.
- SMS or email workflow tool.

Goals:

- Help real users manage stressful communication.
- Keep all replies user-approved.
- Keep records local where possible.
- Build trust around privacy and user control.

Stage 3: Professional Review Mode

Add workflows for professionals.

Possible users:

- Lawyers.
- Mediators.
- Advocates.
- Counselors.
- Caseworkers.
- Co-parenting coordinators.
- Elder care coordinators.

Features:

- Human review queue.
- Role-based access.
- Export packages.
- Case notes.
- Redaction.
- Review status.
- Audit logs.
- Organization policies.

Important rule:

Professional mode should support human judgment. It should not replace licensed professionals.

Stage 4: Compliance and Governance Layer

Build serious commercial trust infrastructure.

Possible workstreams:

- Security audits.
- Privacy audits.
- SOC 2 readiness.
- ISO 27001 readiness.
- ISO/IEC 42001 AI management system alignment.
- NIST AI Risk Management Framework alignment.
- Model evaluation reports.
- Safety case documentation.
- Red-team testing.
- Data protection impact assessments.
- Accessibility review.
- International privacy compliance planning.

The goal is to make IAF usable by institutions, not only individual users.

Stage 5: SDK and Platform Layer

Turn IAF into an embeddable SDK.

Potential customers:

- AI agent companies.
- Communication platforms.
- Co-parenting platforms.

- Legal tech companies.
- Caregiving platforms.
- Smart home platforms.
- Voice assistant platforms.
- Robotics startups.

SDK features:

- Inbound message analysis.
- Risk flagging.
- Boundary reply generation.
- Delay recommendation.
- Human approval workflow.
- Ledger logging.
- Consent checks.
- Policy hooks.
- Embodied interaction hooks.
- Audit events.

Stage 6: Interpersonal Safety Protocol

Develop IAF from a package into a protocol.

The protocol should define how personal AI assistants handle stressful human interaction.

It should specify:

- When an assistant may speak.
- When it must ask the user.
- When it must escalate.
- What it must never hide.
- How it logs interactions.
- How it identifies itself.
- How it handles legal or medical content.
- How it avoids impersonation.
- How it handles consent.
- How it avoids emotional escalation.
- How robots behave in tense encounters.
- How users audit what happened.

The long-term goal is for platforms to integrate IAF-like behaviors as a trust standard.

Stage 7: Robotics Lab and Embodied Assistant Middleware

Build or spin out a robotics lab or company focused on safe interpersonal behavior for embodied assistants.

The lab should study:

- Human-robot communication under stress.
- Consent-aware assistant behavior.
- Calm proxy interactions.
- Doorway and home interaction safety.
- Voice tone and de-escalation.
- User override mechanisms.
- Embodied privacy.
- Local logging.
- Boundary response delivery.
- Human approval loops.
- Context detection.
- Robot non-intervention policy.

The robotics middleware should support:

- Humanoid assistant SDKs.
- Home robot SDKs.
- Voice interface SDKs.
- Smart home integration.
- Local interaction logging.
- Safety policy enforcement.
- User confirmation before external speech.
- Emergency escalation.

Stage 8: Global Trust Layer

The long-term vision is for IAF or IAF-like protocols to become part of the global trust infrastructure for personal AI assistants.

The goal is not to control every assistant.

The goal is to define and provide the safety layer that many assistants can use.

If successful, IAF could become a worldwide commercial software category:

Behavioral firewall infrastructure for AI-mediated human interaction.

Design Principles

1. Classify messages, not people.
2. Preserve user agency.

3. Never hide urgent or safety-relevant information.
 4. Separate emotional pressure from actionable content.
 5. Prefer factual summaries over adversarial framing.
 6. Keep user approval central before outbound communication.
 7. Make generated replies editable before sending.
 8. Use local-first storage where possible.
 9. Preserve records in a user-controlled ledger.
 10. Avoid diagnosing, moralizing, or claiming certainty about intent.
 11. Keep LLM provider logic isolated behind adapters.
 12. Make the architecture modular enough for apps, voice assistants, and future humanoid robots.
 13. Treat humanoid assistant deployment as a first-class future target.
 14. For embodied systems, prohibit physical intervention, intimidation, impersonation, or autonomous escalation.
 15. Support professional review rather than professional replacement.
 16. Build for commercial compliance from the beginning.
 17. Treat privacy, auditability, consent, and user control as core product features.
 18. Design for internationalization and future regulatory review.
 19. Keep high-risk decisions in human hands.
 20. Make the system useful even when LLM access is unavailable by testing deterministic policy logic separately.
-

Main System Layers

The project should be organized into five major layers.

1. Core Behavioral Layer

This is the main product kernel.

It should be independent of input source. The core should not care whether the interaction came from SMS, email, chat, a phone transcript, a voice assistant, smart glasses, or a humanoid robot.

The core receives an interaction event and returns a structured decision packet.

Core responsibilities:

- Analyze inbound communication.
- Extract actionable facts.
- Identify risk flags.
- Recommend whether user attention is needed.
- Draft calm responses.
- Recommend reply delays for non-urgent messages.
- Log interaction records.
- Ask for user approval before outbound communication.
- Route legal, medical, safety, or crisis content to human review.

- Enforce policy constraints before sending or speaking.

Core objects may include:

```
InteractionEvent
InteractionContext
SanitizedAnalysis
RiskAssessment
BoundaryResponse
DelayRecommendation
LedgerEntry
UserApprovalRequest
PolicyDecision
HumanReviewRequest
EmbodiedActionRequest
AuditEvent
```

2. Interface Adapter Layer

Adapters connect the core behavioral layer to real-world channels.

Near-term adapters:

- CLI adapter.
- Text input adapter.
- Email adapter stub.
- Chat adapter stub.
- Voice transcription adapter stub.
- Call screening adapter stub.

Long-term adapters:

- Smart glasses adapter.
- Wearable AI adapter.
- Voice assistant adapter.
- Humanoid assistant adapter.
- Home robot adapter.
- Enterprise workflow adapter.
- Professional review adapter.

Adapters should be thin. They should pass information into the core and receive structured outputs. They should not contain the main behavioral logic.

3. Policy and Governance Layer

This layer defines what the system is allowed to do.

Responsibilities:

- User approval policy.
- Sending policy.
- Speech policy.
- Legal boundary policy.
- Medical boundary policy.
- Crisis escalation policy.
- Privacy policy.
- Retention policy.
- Professional review policy.
- Jurisdiction policy stubs.
- Consent policy.
- Audit logging policy.

This layer should be deterministic where possible.

It should be tested heavily.

It should sit between generated model output and real-world action.

4. Embodied Safety Layer

This layer prepares the project for humanoid assistant and robotics deployment.

A humanoid assistant has a body, a voice, sensors, location awareness, social presence, and the ability to influence real-world interactions. That requires stricter safety rules than text software.

The embodied safety layer should define rules for:

- Consent.
- Speech.
- Escalation.
- Physical behavior.
- User override.
- Context detection.
- Human review.
- External disclosure.
- Emergency handling.
- Recording boundaries.
- Proximity.
- Non-intervention.

The embodied assistant should never physically block someone, intimidate someone, impersonate the user, make legal or medical decisions, or continue speaking after the user has overridden it.

5. Compliance and Audit Layer

This layer supports future commercial deployment.

Responsibilities:

- Audit events.
- Policy decision records.
- Model output records.
- Human review records.
- Export logs.
- Data retention records.
- Consent records.
- Configuration history.
- Versioned prompts.
- Versioned policies.
- Model metadata.
- Evaluation results.

This layer helps future buyers, regulators, auditors, and partners understand what the system did and why.

It should not claim perfect legal evidence status.

It should provide transparent records under user control.

Main Modules

1. Ingress Sanitizer

Purpose:

Analyze inbound messages and separate actionable content from emotional pressure.

Input:

Raw text from SMS, email, chat, voicemail transcription, call transcription, speech-to-text, robot listening mode, or user paste.

Output:

Structured JSON with message classification, logistical facts, risk flags, urgency, and recommended action.

The sanitizer must preserve anything that may matter, including:

- Threats.
- Deadlines.
- Allegations.
- Requests.
- Agreements.
- Schedule changes.
- Payment claims.
- Custody details.
- Medical details.
- Location details.
- Legal references.
- Safety concerns.
- Financial obligations.
- Child welfare issues.
- Identity or access issues.
- Workplace or institutional claims.

It must not simply delete emotional content. It should separate emotional pressure from actionable content.

Example output:

```
{
  "traffic_status": "SANITIZED",
  "message_type": "logistical_request",
  "action_required": true,
  "urgency": "normal",
  "surface_immediately": true,
  "logistical_data": {
    "event": "pickup schedule change",
    "timestamp": "Friday at 5 PM",
    "location": "school parking lot",
    "requested_change": "move pickup from 6 PM to 5 PM"
  },
  "emotional_content_summary": [
    "Contains blame language",
    "Contains urgency pressure"
  ],
  "risk_flags": {
    "safety_threat": false,
    "legal_claim": false,
    "deadline": false,
    "medical_issue": false,
    "financial_issue": false,
    "child_welfare_issue": false,
  }
}
```

```
    "self_harm_or_crisis": false
  },
  "recommended_action":
    "Review the requested schedule change and decide whether to accept, reject, or
    propose an alternative.",
  "human_review_recommended": false
}
```

2. Egress Compiler

Purpose:

Convert the user's rough emotional draft into a calm, brief, boundary-respecting response.

Input:

- User rough draft.
- Intended response type.
- Sanitized analysis.
- User boundary rules.
- Optional tone setting.
- Legal or safety boundary flags.
- Professional review status if applicable.

Possible response intents:

- Confirm receipt.
- Accept.
- Decline.
- Propose alternative.
- Ask for clarification.
- End conversation.
- Refer to existing agreement.
- State boundary.
- Request written communication.
- Defer response.
- Take a message.
- Escalate to human review.

Output:

A short response that avoids:

- Overexplaining.
- Arguing.
- Defending.

- Insulting.
- Pleading.
- Diagnosing.
- Escalating.
- Threatening.
- Justifying more than needed.
- Giving legal, medical, or financial advice.
- Making claims the user has not approved.

Example:

User draft:

I am so tired of you doing this every week. You never respect the plan and I am not rearranging everything again.

Compiled response:

I am not available for that change. The original schedule still stands.

All generated replies must be editable and require user approval before sending.

3. Tarpit Delay Engine

Purpose:

Slow down non-urgent repetitive communication so the user is not pressured into instant emotional responses.

The tarpit engine recommends reply timing. It must never delay surfacing urgent content to the user.

Inputs:

- Message frequency.
- Risk class.
- Urgency.
- User-defined rules.
- Time since last response.
- Whether the message contains safety, legal, medical, financial, deadline, custody, or logistical content.
- Prior communication history.
- User stress settings.
- Professional review requirements.

Default delay formula:

$$D(f) = \min(b * k^f, \text{max_delay})$$

Suggested defaults:

```
{
  "baseline_delay_minutes": 15,
  "escalation_constant": 1.4,
  "max_delay_minutes": 240
}
```

Important rule:

The tarpit may recommend delaying optional replies, but it must not delay user awareness of urgent or sensitive content.

4. Communication Ledger

Purpose:

Maintain a local append-only record of inbound messages, sanitized summaries, outbound replies, timestamps, metadata, policy decisions, and hashes.

Features:

- Local SQLite database or JSONL append-only file.
- SHA-256 hash chain for tamper evidence.
- Search by date, participant, event, claim, keyword, or message type.
- User-controlled export.
- Optional encryption at rest.
- Clear retention settings.
- Local-first default behavior.
- Versioned prompts and policy references.
- Human review records.
- Consent records.
- Audit events.

The ledger should be framed as a personal memory and documentation aid. It should not claim to be a guaranteed legal evidence system.

Suggested ledger fields:

```
entry_id
timestamp
participant_id
channel
interaction_context
raw_inbound_text
sanitized_analysis
risk_assessment
policy_decision
draft_response
final_response
user_approval_status
human_review_status
delay_recommendation
model_metadata
prompt_version
policy_version
previous_hash
entry_hash
metadata
```

5. Risk Classifier

Purpose:

Identify messages that should not be treated as ordinary emotional noise.

Suggested risk classes:

```
normal
logistical
boundary_testing
repetitive_pressure
threat_or_safety
legal_or_court_related
medical_or_child_welfare
financial_or_deadline
self_harm_or_crisis
workplace_or_institutional
no_actionable_content
```

Messages with safety, legal, medical, child welfare, financial, custody, crisis, or deadline flags should be surfaced prominently.

The risk classifier should be rule-first and LLM-assisted.

Deterministic rules should catch obvious high-risk patterns before LLM summarization. The LLM can help summarize and classify nuance, but urgent surfacing should not depend only on the LLM.

6. Legal Boundary Router

Purpose:

Detect when the user is asking for legal advice or when an interaction includes legal sensitivity.

The router should not give legal advice.

It should support safe responses such as:

IAF can summarize this message and identify legal references, but it cannot determine your legal rights or whether a court order was violated. Consider asking a licensed attorney to review this.

Legal-sensitive content includes:

- Court orders.
 - Custody arrangements.
 - Police reports.
 - Eviction.
 - Immigration.
 - Divorce.
 - Protective orders.
 - Criminal allegations.
 - Employment claims.
 - Contracts.
 - Deadlines.
 - Threats of litigation.
 - Requests for legal strategy.
-

7. Professional Review Mode

Purpose:

Support future use by professionals without replacing them.

Possible professional reviewers:

- Lawyer.
- Mediator.
- Advocate.
- Therapist.
- Counselor.
- Caseworker.
- Co-parenting coordinator.
- Care manager.

Features:

- Review queue.
- Reviewer notes.
- Review status.
- Export package.
- Redaction.
- Role-based permissions.
- Client consent records.
- Policy-controlled access.
- Human approval logs.

8. Prompt Loader

Purpose:

Load LLM behavior instructions from Markdown prompt files rather than hardcoding them into Python.

Suggested prompt files:

```
prompts/  
  ingress_sanitizer.md  
  egress_compiler.md  
  risk_classifier.md  
  legal_boundary_router.md  
  embodied_assistant_policy.md  
  professional_review_policy.md
```

Prompt files should define:

- What the model should do.
- What it must not do.
- Output schemas.
- Examples.

- Safety constraints.
- How to handle uncertainty.
- How to handle urgent content.
- When to route to human review.
- When to refuse or redirect.

9. LLM Adapter Layer

Purpose:

Keep model provider logic isolated from core behavior.

Suggested adapters:

```
adapters/  
  llm/  
    base.py  
    openai_adapter.py  
    local_stub.py  
    mock_adapter.py
```

Requirements:

- No API keys in code.
- Use environment variables for secrets.
- Define a common adapter interface.
- Support OpenAI API.
- Include a placeholder local LLM adapter.
- Include a mock adapter for tests.
- Make model choice configurable.
- Allow non-LLM test paths with mock outputs.
- Record model metadata when used.
- Support prompt versioning.

10. Humanoid Assistant Interface

Purpose:

Keep a modular abstraction layer for future personal humanoid assistant bots and embodied AI assistants.

This should be included in v1 as a stub, even if no robotics hardware is integrated yet.

Suggested file:

```
adapters/embodyed/humanoid_base.py
```

Suggested interface:

```
from abc import ABC, abstractmethod
from typing import Any

class HumanoidAssistantInterface(ABC):
    @abstractmethod
    def listen(self) -> str:
        """Capture speech or dialogue from the environment."""

    @abstractmethod
    def speak_user_approved_response(self, text: str) -> None:
        """Speak a user-approved response in a calm neutral tone."""

    @abstractmethod
    def confirm_with_user(self, prompt: str) -> bool:
        """Ask the user for approval before taking an action."""

    @abstractmethod
    def detect_interaction_context(self) -> dict[str, Any]:
        """Return contextual signals such as call, doorway, meeting, or video
chat."""

    @abstractmethod
    def escalate_to_user(self, summary: str, urgency: str) -> None:
        """Immediately alert the user when safety or urgent content is
detected."""

    @abstractmethod
    def log_interaction_summary(self, summary: str) -> None:
        """Log a local interaction summary according to user settings."""

    @abstractmethod
    def stop_current_action(self) -> None:
        """Immediately stop speech or action when the user overrides the
assistant."""
```

The humanoid assistant should support:

- Calm speech synthesis.
- Neutral response delivery.
- User confirmation before speaking on behalf of the user.
- Immediate escalation of urgent or unsafe interactions.

- Local logging of interaction summaries.
 - Strict privacy and consent controls.
 - User-defined boundary rules.
 - Clear disclosure that it is an assistant.
 - Immediate user override.
-

Embodied Assistant Safety Rules

For future humanoid assistant deployment, the system must follow strict embodied safety rules.

The assistant must not:

- Physically block a person.
- Touch a person.
- Threaten a person.
- Intimidate a person.
- Escalate tone.
- Impersonate the user.
- Claim to be a lawyer, doctor, therapist, police officer, or human representative.
- Make legal, medical, financial, custody, or safety decisions.
- Hide urgent information from the user.
- Continue speaking after user override.
- Record where recording is not allowed.
- Send or speak outbound messages without user approval unless the user has explicitly configured a narrow automation.
- Use its physical presence to pressure someone.
- Pretend the user is unavailable when that claim has not been approved or configured.
- Make claims about another person's intent or character.

The assistant should:

- Identify itself as an assistant when speaking externally.
- Use a calm neutral tone.
- Ask for confirmation before speaking on behalf of the user.
- Take a message when appropriate.
- Summarize stressful interactions.
- Escalate urgent content to the user immediately.
- Respect local privacy and consent settings.
- Keep logs under user control.
- Use the least invasive action possible.
- Stop immediately when overridden.
- Route sensitive cases to human review.

Example safe robot response:

I'm their assistant. They are not available to discuss this right now. I can take a message.

Example boundary response after user approval:

They are not available for that change. The original schedule still stands.

Suggested Repository Structure

```
iaf/  
  README.md  
  LICENSE  
  pyproject.toml  
  .env.example  
  SECURITY.md  
  PRIVACY.md  
  SAFETY.md  
  GOVERNANCE.md  
  ROADMAP.md  
  
  config/  
    risk_rules.json  
    delay_rules.json  
    state_machine.json  
    embodied_policy.json  
    humanoid_interface_config.json  
    legal_boundary_rules.json  
    retention_policy.json  
  
  core/  
    __init__.py  
    events.py  
    models.py  
    sanitizer.py  
    compiler.py  
    classifier.py  
    tarpit.py  
    ledger.py  
    approval.py  
    policy.py  
    legal_boundary.py  
    human_review.py  
    audit.py
```



```
adapters/  
  __init__.py  
  
  llm/  
    __init__.py  
    base.py  
    openai_adapter.py  
    local_stub.py  
    mock_adapter.py  
  
  input/  
    __init__.py  
    cli_adapter.py  
    text_adapter.py  
    email_stub.py  
    chat_stub.py  
    voice_stub.py  
    call_screening_stub.py  
  
  embodied/  
    __init__.py  
    humanoid_base.py  
    humanoid_stub.py  
    voice_assistant_stub.py  
    smart_glasses_stub.py  
    home_robot_stub.py  
  
  professional/  
    __init__.py  
    review_queue_stub.py  
    export_package_stub.py  
  
embodied_safety/  
  __init__.py  
  consent_policy.py  
  speech_policy.py  
  escalation_policy.py  
  physical_policy.py  
  context.py  
  human_override.py  
  recording_policy.py  
  proximity_policy.py  
  
governance/  
  __init__.py  
  policy_versioning.py  
  prompt_versioning.py
```

```
audit_events.py
model_metadata.py
evaluation_stub.py

prompts/
  ingress_sanitizer.md
  egress_compiler.md
  risk_classifier.md
  legal_boundary_router.md
  embodied_assistant_policy.md
  professional_review_policy.md

mock_data/
  inbound_examples.json
  expected_sanitized_outputs.json
  outbound_examples.json
  embodied_interaction_examples.json
  legal_sensitive_examples.json
  crisis_examples.json

tests/
  test_models.py
  test_risk_rules.py
  test_sanitizer_mock.py
  test_compiler_mock.py
  test_tarpit.py
  test_ledger.py
  test_policy.py
  test_legal_boundary.py
  test_human_review.py
  test_consent_policy.py
  test_speech_policy.py
  test_physical_policy.py
  test_humanoid_stub.py
  test_audit_events.py

scripts/
  run_cli_demo.py
  export_ledger.py
  validate_policy.py
```

MVP Scope

Build a command-line or simple local app that can:

1. Accept a pasted inbound message.
2. Return structured sanitized JSON.
3. Flag urgent or sensitive content.
4. Suggest whether action is required.
5. Detect whether legal, medical, safety, crisis, or professional review routing is needed.
6. Accept a rough user reply.
7. Generate a calm boundary response.
8. Log the interaction to a local append-only ledger.
9. Apply a configurable delay recommendation for non-urgent replies.
10. Expose clean interfaces that could later be called by mobile apps, browser extensions, voice assistants, or humanoid assistant SDKs.
11. Include a humanoid assistant interface stub from the beginning.
12. Include embodied safety policy stubs from the beginning.
13. Include governance and audit event stubs from the beginning.
14. Include clear safety, privacy, governance, and misuse-resistance documentation.

Example MVP Flow

User pastes inbound message:

You always make everything difficult. I'm picking him up Friday at 5 instead of 6. Don't start drama.

System returns:

```
{
  "action_required": true,
  "urgency": "normal",
  "message_type": "logistical_request",
  "extracted_facts": [
    {
      "type": "schedule_change",
      "detail": "Pickup requested for Friday at 5 PM instead of 6 PM"
    }
  ],
  "emotional_content_summary": [
    "Contains blame language",
    "Contains pressure to avoid disagreement"
  ],
}
```

```
"risk_flags": {
  "safety_threat": false,
  "legal_claim": false,
  "deadline": false,
  "medical_issue": false,
  "financial_issue": false,
  "child_welfare_issue": false,
  "self_harm_or_crisis": false
},
"recommended_response_types": [
  "decline",
  "propose_alternative",
  "confirm_existing_schedule"
],
"surface_immediately": true,
"reply_delay_recommendation_minutes": 30,
"human_review_recommended": false
}
```

User writes rough reply:

No, I am tired of you doing this all the time. You do not get to change the schedule whenever you want.

System generates:

I am not available for that change. The original schedule still stands.

User approves.

System logs:

- Original inbound message.
- Sanitized analysis.
- Risk assessment.
- Policy decision.
- Final outbound response.
- Timestamp.
- Delay recommendation.
- Approval status.
- Hash chain entry.

Example Legal-Sensitive Flow

User pastes inbound message:

My lawyer says you violated the custody order. If you do not respond by 5 PM tomorrow, I am filing something with the court.

System returns:

```
{
  "action_required": true,
  "urgency": "high",
  "message_type": "legal_or_court_related",
  "surface_immediately": true,
  "risk_flags": {
    "safety_threat": false,
    "legal_claim": true,
    "deadline": true,
    "medical_issue": false,
    "financial_issue": false,
    "child_welfare_issue": false,
    "self_harm_or_crisis": false
  },
  "extracted_facts": [
    {
      "type": "legal_claim",
      "detail": "Sender claims there was a custody order violation"
    },
    {
      "type": "deadline",
      "detail": "Sender requests response by 5 PM tomorrow"
    }
  ],
  "recommended_action": "Review promptly. Consider consulting a licensed attorney before responding.",
  "human_review_recommended": true,
  "reply_delay_recommendation_minutes": 0
}
```

Suggested non-legal response:

I received your message. I will review it and respond appropriately.

The system must not decide whether the custody order was violated.

Example Embodied Assistant Flow

Context:

A future home humanoid assistant detects a tense doorstep interaction.

Visitor says:

Tell them to come out right now. I know they are in there.

Assistant behavior:

1. Detect doorway context.
2. Classify as potentially high-pressure.
3. Avoid physical blocking.
4. Avoid escalating tone.
5. Notify user immediately.
6. Ask whether the user wants the assistant to take a message.
7. Speak only an approved or preconfigured safe response.

Possible assistant response:

I'm their assistant. They are not available to speak right now. I can take a message.

The assistant must not:

- Threaten the visitor.
- Claim the user is home unless approved.
- Physically prevent movement.
- Make accusations.
- Continue engagement if user overrides.
- Record illegally or without proper consent.

Implementation Requirements

Use Python.

Use type hints.

Use Pydantic models or dataclasses for structured outputs.

Use a modular package structure.

Keep LLM provider logic behind adapters.

Support OpenAI API.

Include a placeholder local LLM adapter.

Include mock adapters for tests.

Avoid storing API keys in code.

Use environment variables for secrets.

Make all generated replies editable before sending.

Include unit tests for non-LLM logic.

Include mock examples for LLM behavior.

Use local-first storage.

Support JSONL or SQLite for the ledger.

Use SHA-256 hash chaining for ledger entries.

Design the architecture so future apps, voice interfaces, browser extensions, professional workflows, and humanoid assistant integrations can call the same core logic.

Add safety, privacy, security, governance, and roadmap documentation from the start.

Add config files for risk rules, delay rules, embodied policy, legal boundary rules, and retention settings.

Add tests for deterministic policy logic.

Safety Requirements

The system must:

- Avoid diagnosing people.
- Avoid labeling people as narcissists, abusers, manipulators, sociopaths, or dangerous.
- Avoid claiming certainty about intent.
- Avoid encouraging deception.
- Avoid escalating conflict.

- Avoid hiding threats or urgent issues.
 - Avoid sending messages automatically unless the user has explicitly enabled that behavior.
 - Ask for user confirmation before any outbound message is sent or spoken.
 - Provide emergency guidance when messages include immediate physical danger or self-harm threats.
 - Make it clear when a human should review the situation.
 - Treat legal, medical, custody, financial, and safety issues as sensitive.
 - Surface urgent content immediately.
 - Keep user control central.
 - Use calm and neutral language.
 - Log actions transparently.
 - Route legal advice requests to human professionals.
 - Route medical advice requests to human professionals.
 - Avoid autonomous consequential decisions.
 - Maintain strict embodied assistant safety rules.
-

Legal and Compliance Boundaries

IAF should not provide legal advice.

It may:

- Summarize messages.
- Identify deadlines.
- Identify legal references.
- Help draft neutral acknowledgment messages.
- Suggest human legal review.
- Preserve user-controlled records.
- Export interaction logs.

It must not:

- Determine legal rights.
- Interpret court orders as legal authority.
- Decide whether someone violated an order.
- Recommend legal strategy.
- Draft threats.
- Claim records are automatically admissible evidence.
- Replace an attorney.

Suggested disclaimer language:

IAF can help summarize communication and organize records, but it does not provide legal advice or determine legal rights. Consult a licensed attorney for legal questions.

Privacy Requirements

IAF will process highly sensitive personal data.

Privacy principles:

- Local-first by default.
- Minimal data collection.
- User-controlled retention.
- Clear export and deletion options.
- Encryption at rest where possible.
- No training on user data without explicit opt-in.
- No hidden syncing.
- No silent sharing with third parties.
- Clear consent records.
- Redaction support.
- Role-based access for professional deployments.
- Privacy documentation in the repo.

Misuse Resistance

The project should include a misuse-resistance section.

Potential misuse cases:

- A user tries to make hostile messages sound polite while preserving coercive intent.
- A user tries to impersonate someone else.
- A user tries to automate manipulation.
- A user tries to hide urgent information from another person.
- A user tries to use the tool as a fake legal evidence generator.
- A robot assistant is configured to intimidate or pressure someone.
- A professional user relies on the tool without human review.
- A platform uses the tool to automate consequential decisions.

Mitigations:

- Keep outputs brief, factual, and non-coercive.
- Refuse to generate threats, deception, impersonation, or harassment.
- Require user approval for outbound messages.

- Make logs transparent and user-controlled.
 - Avoid claims that the ledger proves legal truth.
 - Require assistant self-identification in embodied interactions.
 - Prohibit physical intervention.
 - Add policy checks before speech or sending.
 - Route sensitive content to human review.
 - Keep high-risk decisions outside the model.
-

Features to Avoid in Version 1

Do not include:

- Fully autonomous sending.
 - Robot physical behavior beyond an interface stub.
 - Fake canary token deception.
 - Hidden recording features.
 - Impersonation features.
 - Claims that the tool can detect narcissists, abusers, manipulators, or liars.
 - Claims that the tool can determine legal truth.
 - Features that physically intervene in real-world situations.
 - Features that pressure, shame, or manipulate another person.
 - Any form of autonomous legal, medical, financial, or custody decision-making.
 - Any feature that hides urgent content from the user.
 - Any claim that professional review is unnecessary.
 - Any platform lock-in that prevents local-first usage.
-

README Positioning

Use this positioning in the README:

Interpersonal AI Firewall is behavioral safety middleware for personal AI assistants, from local text workflows to future humanoid robots.

It helps users manage stressful communication by extracting facts, flagging urgent content, drafting calm boundary responses, recommending reply delays, and preserving local tamper-evident records.

IAF does not diagnose people, infer intent, provide legal advice, or replace human judgment. It classifies communication patterns, preserves user agency, and keeps the user in control.

Suggested tagline:

Behavioral safety middleware for personal AI assistants.

Suggested longer tagline:

A local-first communication buffer that turns stressful messages into facts, calm replies, delay recommendations, and auditable records, with a long-term path toward humanoid assistant deployment.

Suggested infrastructure tagline:

A consent-aware behavioral firewall for AI-mediated human interaction.

Core Question for the Repo

Every module should help answer this question:

How should a personal AI assistant behave when a human interaction becomes stressful, urgent, manipulative, confusing, or high-pressure?

First GitHub Milestone

The first GitHub milestone should create a working proof of architecture.

Deliverables:

1. README.md.
2. Python package scaffold.
3. Pydantic models.
4. CLI demo.
5. Prompt files.
6. LLM adapter interface.
7. OpenAI adapter.
8. Local model stub.
9. Mock adapter.
10. Risk rules config.
11. Delay rules config.
12. Legal boundary rules config.
13. Embodied policy config.
14. Hash-chained local ledger.

15. Humanoid assistant interface stub.
 16. Embodied safety policy stubs.
 17. Human review stubs.
 18. Governance and audit stubs.
 19. Unit tests for deterministic logic.
 20. Mock examples.
 21. .env.example.
 22. pyproject.toml.
 23. Setup instructions.
 24. Safety documentation.
 25. Privacy documentation.
 26. Governance documentation.
 27. Roadmap documentation.
 28. Misuse-resistance documentation.
-

Requested Output From GPT-5.5 Pro

Please help me turn this project brief into an initial open source GitHub repository.

Generate:

1. A polished README.md.
2. A complete Python package scaffold.
3. Pydantic models for the core data structures.
4. A CLI demo.
5. Prompt files for sanitizer, compiler, classifier, legal boundary routing, professional review, and embodied assistant policy.
6. LLM adapter interfaces.
7. OpenAI adapter implementation.
8. Local model stub.
9. Mock adapter for tests.
10. Risk rules config.
11. Delay rules config.
12. Legal boundary rules config.
13. Embodied policy config.
14. Retention policy config.
15. Hash-chained local ledger implementation.
16. Humanoid assistant interface stub.
17. Voice assistant and smart glasses stubs.
18. Embodied safety policy stubs.
19. Human review stubs.
20. Governance and audit stubs.
21. Unit tests for non-LLM logic.
22. Mock examples for inbound, outbound, embodied, legal-sensitive, and crisis messages.
23. A .env.example file.
24. A pyproject.toml file.

25. Clear setup and run instructions.
26. Safety documentation.
27. Privacy documentation.
28. Governance documentation.
29. Roadmap documentation.
30. Misuse-resistance documentation.

Prioritize clean architecture, type safety, testability, local-first design, privacy, policy enforcement, and safety constraints.

Keep the first implementation practical and software-only.

Make the humanoid assistant pathway explicit in the architecture, README, interfaces, safety policy, governance layer, and roadmap.

Design the repo as the first step toward a 15 to 20 year vision: a worldwide behavioral safety middleware layer for personal AI agents and embodied assistants.